Bit-Exact Software Modelling of IEEE 754 binary32 Arithmetic with 32-Million-Case Differential Verification

Shayan Amouzadeh

Technical Report · Version 1.0 · October 2026

Code: https://github.com/Shayan-Amz/Floating-Point-ALU (MIT) · DOI: to be assigned by Zenodo

Abstract

We describe a header-only C++17 software model of an IEEE 754-2019 single-precision (binary32) arithmetic unit implementing addition, subtraction, multiplication and division with integer arithmetic only. The datapath follows the classical hardware decomposition — unpack, align/multiply/long-divide with a guard–round–sticky (GRS) extension, normalise, round, pack — and supports all four IEEE rounding directions, subnormal operands and results, signed zeros, infinities and quiet/signalling NaN propagation. Because the entire datapath is `constexpr`, results can be checked at compile time. Correctness is established by differential testing against the host floating-point unit under `fesetround()`: 32,026,992 operand-pair checks (a 41 × 41 grid of hand-picked edge values plus 250,000 random pairs from each of eight stress distributions, each evaluated for four operations and four rounding modes) produce zero bit-level mismatches on GCC, Clang, AppleClang and MSVC. A companion harness quantifies the defects of the author's original course implementation (58.2 % bit-exact, 40.8 % off by one ulp, 1.1 % worse on 1.4 M random pairs) and attributes them to missing sticky-bit alignment, truncation instead of round-to-nearest-even, and absent gradual underflow — all eliminated in the rewrite.

Keywords: IEEE 754, floating-point arithmetic, guard–round–sticky rounding, subnormals, differential testing, constexpr, C++17.

1. Introduction

Floating-point units are among the most error-prone components of a processor, and software models of them are a standard teaching and verification tool. The usual undergraduate exercise — implement a binary32 adder — is typically accepted when it produces plausible results; it is rarely checked bit-for-bit against a reference. This report documents a rewrite of such a course project into a model that is (i) complete with respect to the IEEE 754 binary32 operations add/sub/mul/div, (ii) bit-exact in all four rounding modes, and (iii) verified at a scale (3.2×10^7 checks) that makes residual rounding errors very unlikely.

Contributions:

1. A compact (355-line), dependency-free, `constexpr` C++17 implementation of the binary32 add/sub/mul/div datapath with explicit GRS rounding (Section 3).
2. A differential conformance suite against the host FPU with 32,026,992 checks and a 4-compiler CI matrix (Section 4).
3. A quantitative post-mortem of the legacy adder's error distribution (Section 5).

2. Background: binary32 and rounding

A binary32 value packs a sign bit, an 8-bit biased exponent (bias 127) and a 23-bit fraction; normal numbers carry an implicit leading one, giving a 24-bit significand. Exponent field 0

denotes zero or a subnormal (no implicit one, exponent −126); field 255 denotes ±∞ (fraction 0) or NaN (fraction ≠ 0, quiet bit = fraction MSB).

Correct rounding requires that the infinitely precise result be rounded once to 24 bits. In hardware this is achieved by carrying three extra bits beyond the 24-bit significand — guard (G), round (R) and sticky (S, the OR of all further discarded bits) — which are sufficient to decide every rounding direction exactly: round-to-nearest-even (RNE), toward zero (RTZ), toward +∞ (RTP) and toward −∞ (RTN).

3. Design

The model lives in `include/fp32/fp32.hpp`. Values are a `struct` of unpacked fields plus free functions; constants are `kFractionBits = 23`, `kExponentBits = 8`, `kBias = 127`. Rounding mode is an `enum class Rounding { NearestEven, TowardZero, TowardPositive, TowardNegative }` parameter to every operation.

3.1 Addition and subtraction

`sub(a, b)` is `add(a, negate(b))`. `add` orders the operands so that |x| ≥ |y| (the result takes the sign of the larger magnitude), widens both significands by three GRS bits, and aligns the smaller with `shift_right_sticky()`, which OR-reduces every bit shifted out into the sticky position. Magnitudes are added or subtracted; a carry-out is handled by a one-bit right shift (again with sticky), and leading-zero cancellation by a left normalisation that is bounded at exponent 1 — this single bound is what makes gradual underflow fall out naturally without a separate subnormal path. Exact cancellation yields a signed zero whose sign depends on the rounding mode (−0 only under RTN), as required by IEEE 754 §6.3.

3.2 Multiplication

`detail::normalize()` first renormalises subnormal operands (shifting the leading one into place and adjusting the exponent). The two 24-bit significands are multiplied into a 48-bit `uint64_t`; the leading one lands at bit 46 or 47 and is detected to set the exponent. If the resulting exponent is below 1, the product is right-shifted (with sticky) into subnormal range before rounding.

3.3 Division

The dividend is doubled when necessary so that the quotient lies in [1, 2). An integer long division `(dividend << 26) / y.significand` produces 27 quotient bits (24 significant + G + R + one spare); a non-zero remainder sets sticky. This is the only place where a hardware implementation would differ materially (iterative SRT/Newton), but the rounding behaviour is identical.

3.4 Rounding and packing

`detail::round_and_pack()` operates on a uniform layout (bit 26 = leading one, bits 25..3 = fraction, bits 2..0 = G/R/S). It applies the selected rounding rule, fixes up the 1.111…→10.000 carry, packs subnormals and zeros, and resolves overflow per IEEE 754 §7.4: ±∞ under RNE, or the largest finite magnitude when the direction mode points toward zero. NaN handling (`propagate_nan`) preserves the payload of the first NaN operand and sets the quiet bit; invalid operations (∞−∞, 0×∞, 0/0, ∞/∞) return the default NaN `0x7FC00000`.

3.5 Compile-time evaluation

Every function is `constexpr`; the test suite contains `constexpr bits_t third = div(0x3F800000u, 0x40400000u); static_assert(third == 0x3EAAAAABu)`, so a regression in the datapath is a compile error.

4. Verification

4.1 Method

The reference is the host FPU. The test binary is compiled with `-frounding-math` (GCC/Clang) or `/fp:strict` (MSVC) so that the compiler neither constant-folds nor reorders the reference operations, and `fesetround()` is called before each reference evaluation. For each operand pair (a, b), each operation $\circ \in \{+, -, \times, \div\}$ and each rounding mode m, the suite asserts

bits(model$\circ$(a, b, m)) = bits(host$\circ$(a, b)) under m,

where any two NaN results are treated as equal (NaN sign and payload are implementation-defined and are deliberately not asserted).

4.2 Operand sets

- **Hand-picked grid.** The 41-entry table `kInteresting` (±0, ±min subnormal, two further subnormals, ±min normal, ±1, 1 + ulp, just-under-2, 2, 3, π, 0.1, 0.2, 1/3, 2^{23}, 2^{24}, 2^{127}, ±max finite, ±∞, quiet/signalling/negative NaN, 2^{-24}…2^{-22}, assorted magnitudes) in a 41 × 41 cross product, plus 6 regression pairs that broke the original program (1.5 + (−1.5) infinite loop, 0.1 + 0.2 one ulp low, 1e-40 + 1e-40, 2^{24} + 1 and 2^{24} + 1 + 1 ties, 3/1 vs 1/3): (1,681 + 6) × 4 ops × 4 modes = **26,992 checks**.
- **Random families.** A seeded `std::mt19937_64` (seed `0x5EED`, so runs are reproducible) draws 250,000 pairs from each of eight generators — `any_bits`; `normal` (exponent 1–254); `near_one` (exponent 120–134, short alignment shifts); `tiny` (exponent 0–30, underflow territory); `huge` (exponent 220–254, overflow territory); `subnormal` × `normal`; `huge` ÷ `tiny`; and `cancelling` (opposite-sign operands within ±4 ulp) — each pair checked for 4 ops × 4 modes: 250,000 × 8 × 16 = **32,000,000 checks**.
- **Properties.** 15 hardware-independent `expect` checks (1.5 + (−1.5) = +0, and −0 under RTN; ∞ − ∞, 0 × ∞, 0/0 → default NaN; 1/±0 → ±∞; max + max → +∞ under RNE but saturates to max under RTZ; min_subnormal × 0.5 → +0 and 3·min_subnormal × 0.5 → 2·min_subnormal (ties-to-even in the subnormal range); 1 − 1 = +0; binary-string parse/format round trip; malformed string rejected) and 2 `static_asserts` (1 + 1 = 2 and 1/3 = `0x3EAAAAAB` evaluated at compile time).

4.3 Results

Suite	Checks	Failures	Wall time
Hand-picked grid + regressions	26,992	0	—
Random, 8 families × 250 k × 16	32,000,000	0	—
Properties + static_asserts	17	0	—
Total (`./test_fp32 250000`)	**32,026,992**	**0**	≈ 0.8 s (g++ -O2, x86-64)

The suite is run as CTest targets `conformance` (250 k per family) and `conformance_quick` (20 k) on a 4-job GitHub Actions matrix — ubuntu/gcc, ubuntu/clang, macos/AppleClang, windows/MSVC — all green at the time of writing. The CLI `fp32calc` (with `--trace` showing every datapath stage and `--mode {even,zero,up,down}`) is smoke-tested in the same workflow.

5. Post-mortem of the legacy adder

The original course submission (`legacy/Project_Final.cpp`, kept for reference) only implements addition and is not bit-exact on cancellation, so the `legacy_accuracy` tool compares it with the host FPU on addition of random normal operands under RNE,

stratified by exponent gap: 14 gaps {0, 1, 2, 3, 4, 6, 8, 12, 16, 20, 23, 24, 25, 26} × 100,000 pairs = 1.4 million pairs. Pooled over all gaps:

Outcome	Share
Bit-exact	58.2 %
Off by exactly 1 ulp	40.8 %
Worse than 1 ulp (incl. wrong zero sign / missing subnormals)	1.1 %

Root causes identified from the code: (a) alignment shifts discarded bits without a sticky OR, so ties were mis-detected; (b) the result was truncated rather than rounded to nearest-even; (c) no gradual-underflow path; (d) x + (−x) entered an infinite normalisation loop. The rewrite (`fp32::add`) scores 100 % in every gap bucket on the same tool. `tools/plot_accuracy.py` reproduces the per-gap figure (`docs/figures/legacy_vs_fp32.png`).

6. Limitations

- binary32 only — no binary64/binary16, no FMA, no square root, no conversions beyond float ↔ bits and binary-string I/O.
- Status flags (inexact, underflow, overflow, invalid, divide-by-zero) are not modelled; only the result bits are.
- Division is integer long division, not a hardware-realistic iterative algorithm; cycle/latency behaviour is out of scope.
- The host FPU is the oracle; NaN sign/payload are not asserted. For the four basic operations correctly rounded results are mandated by the standard, so SSE/NEON hardware is an adequate reference.
- The legacy comparison covers addition only (the legacy program implemented no other operation).

7. Reproducibility

```
git clone https://github.com/Shayan-Amz/Floating-Point-ALU
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build
ctest --test-dir build -R conformance        # 32,026,992 checks
./build/legacy_accuracy                      # 58.2 % / 40.8 % / 1.1 % table
```

Verified on the GitHub Actions `ubuntu-latest` (GCC, Clang), `macos-latest` (AppleClang) and `windows-latest` (MSVC / VS 2022) images; independently re-run by a third party on 2026-10 with `g++ -std=c++17 -O2 -frounding-math` (32,026,992 checks, 0 failures, 0.8 s). No third-party dependencies.

References

1. IEEE Std 754-2019, IEEE Standard for Floating-Point Arithmetic, 2019.
2. D. Goldberg, "What every computer scientist should know about floating-point arithmetic," ACM Computing Surveys 23(1), 1991.
3. J.-M. Muller et al., Handbook of Floating-Point Arithmetic, 2nd ed., Birkhäuser, 2018.
4. ISO/IEC 14882:2017, Programming Languages — C++ (C++17).

Licence of this report: CC BY 4.0. Source code: MIT.